Forms in Symfony

Install: composer require symfony/form

Concept: we talked about **FormType**, every thing in forms is a *FormType*, like buttons textarea.. all also called FormType, select is a FormType

The whole form is also called FormType

FormType is a *class* (service) that extends a class called **AbstractType**

FormType has access a specific class called **FormBuilder**, to use it to construct forms

When it comes to forms in symfony, it works in a bidirectional way: you can either go from FormType object to html form, or from form submissions from html form to Formtype object.

Create forlder src/Form/Type

Create a class calledf ContactUsType [Now to make a class to be as a form and gain access to the form builder, you need to extends from AbstractTyupe]:

```
class ContactUsType extends AbstractType {}
```

AbstractType has many useful methods that we can use to build the form like: buildForm, buildView, configureOptions...

Now lets deal with **buildForm**: this is used to construct the form

```
class ContactUsType extends AbstractType {
    public funcxtion buildForm(FormBuilderInterface $builder, array $options): void {
        $builder->add() # add a field to our form
        $builder->add("emailAddress")
                  ->add("fullName")
                  ->add("message");
    }
}
```

Now that we have this FormType representing the form, now we will use it and display it

-> Create a controller

```
class ContactUsController extends AbstractController {
    public function index() {
        # Preferable: validate and display it should be in
        # the same action to make maintainability easier

        # createForm needs the FormType that represent our form as argument type
        $form = $this->createForm(type: ContactUsType::class);

        return $this->render('contact/index.html.twig', [
            "contact_us" => $conmtactUs■
        ]);
    }
}
```

Now go and create a twig template to render this form

(Leaving action empty send the form to the same action that renders it)

```
#twig file
<h1>Contact us</h1>
Your review intereseted us
<form method="post">
    <div>
        <label for="full_name">Full name: </label>
        <input type="text" name="fullName" id="full_name" />
    </div>

    <div>
        <label for="email_address">Email address: </label>
        <input type="email" name="emailAddress" id="email_address" />
    </div>

    <div>
        <label for="message">Email address: </label>
        <textarea name="message" id="message"></textarea>
    </div>

    <button type="submit">Envoyer</button>
</form>
```

we need a way to get the above form using FormType not typing the form ourselves

-> to do so, we can use helpers like form(), form_start(), form_end, form_label... and all these helpers receive the form instance of FormType like {{ form($form) }}

Display form : {{ form(contact_us) }}

if we do the above in a twig file, we will get a form html generateg automatically by symfony transformers along aith a hidden token for csrf

The csrf configuration is done in config/packages/csrf.yaml: try to comment this

```
framework
    csrf_protection
        stateless_token_ids:
            # - submit
```

then refresh and you will see that the form will no longer generate the hidden input to handle the csrf since you disable it

If it is enabled (by default it is enabled), snfony generate a csrf token and store it in a session, and check for the next request to come if it has the csrf token submitted and match the one in the sesion to validate that the user submit the sata fromn the form

You can handle this per form to not make the config in yaml file apply to all forms in the app. This is possible using **configureOptions** in your form class:

Search for handle csrf token per form in symfony for more informations and docs: (you can handle enable/disable csrf, change its name shown in forms)

```
class TaskType extends AbstractType
{
    // ...
    public function configureOptions(OptionsResolver $resolver): void
    {
        $resolver->setDefaults([
            'data_class'      => Task::class,
            'csrf_protection' => true,
```

```
            'csrf_field_name' => 'custom_token_name',
            'csrf_token_id'   => 'task_item',
        ]);
    }
    // ...
}
```

csrf_field_name: control the value generated within input name attribute:

```
framework:
    csrf_protection:
        field_name: my_custom_token
```

Now the form will generate:

```
<input type="hidden" name="my_custom_token" value="..." />
```

csrf_token_id

`csrf_token_id` defines the token "intention" (context).

It tells Symfony:

> "This token belongs to THIS specific form or action."

It is used internally to generate and validate the token. Example:

```
$builder->setMethod('POST')
    ->setAction('/submit')
    ->add('_token', HiddenType::class,
    [
        'mapped' => false,
        'data' => $csrfTokenManager>getToken('contact_form'),
    ]);
```

Here: `'contact_form'` is the **csrf_token_id**.

When validating:

```
$csrfTokenManager->isTokenValid( new  CsrfToken('contact_form', $submittedToken));
```

The `'contact_form'` must match.

Please note: if you use `{{ form(contact_us) }}`, you cannot control your form, so you need to use another way of declaring your form using the following:

```
{{ form_start(form) }}
    // here is form types (form widgets like inputs and textareas)
{{ form_end(form) }}
```

There is what is called **form_row**: form_row can be applicable on the whole form as well as to each form widget; It renders all of this together:

- ``

- validation errors

- the (widget)

- help text (if defined)

example:

```
{{ form_row(form.email) }}
```

So internally it is roughly equivalent to:

```
{{ form_label(form.email) }}
{{ form_errors(form.email) }}
{{ form_widget(form.email) }}
{{ form_help(form.email) }}
```

In short:

form_row() = render one full field (label + input + errors + help) in one line.

if we do : `{{ form_row(form) }}`

we will have all parts of form_row like label, validatioin and help text ... to the whole form so the parts of form_row applied to the whole form just like when you did it with email. Example:

```
{{ form_start(form) }}

    {{ form_row(contact_us) }}
    <button type="submit">Send</button>
{{ form_end(form) }}
```

Here if you do this, you will notice trhat the form generated will always give text type to the fields, so what you need to do is to control you formTypes one by one. To do so, we can specify thetype within buildForm inside the FormType class:

```
public function buildForm
    (FormBuilderInterface $builder, array $options): void
{
    $builder
        ->add('username', TextType::class)
        ->add('email', EmailType::class)
    ;
    # namespaceL use Symfony\Component\Form\Extension\Core\Type\EmailType;
}
```

Now if you refresh the page, you will see the template reflect changes in form controls

Now if we want the an input to be optional and not required, we can configure it by adding options to it like following:

```
$builder
    ->add('username', TextType::class, options: [
        "required" => false,
        "disabled" => true,
```

```
            # for other unsupported attributes, you can use attr
            "attr" => [
                "class" => "red",
                "placeholder" => "Your shit here..."
            ]
        ])
        ->add('email', EmailType::class)
    ;
```

to get options, you can go to symfony form supported field types for more..

attr is used to attribute inputs while label_attr is used to attribute labels (attributing is done with attr only for attributes that are not given bby symfony by default like placeholder is not present within the list of attributes so we can add attr to define placeholders for your inputs)

```
    $builder
        ->add('username', TextType::class, options: [
            "required" => false,
            "disabled" => true,
            # for other unsupported attributes, you can use attr
            "attr" => [
                "class" => "red",
                "placeholder" => "Your shit here..."
            ],
            "label_attr" => [
                "class" => "red",
                "placeholder" => "Your shit here..."
            ]
        ])
    ;
```

Up until now, even though we confiure our fields one by one, we still using form_row(form) and that shows everything, now we need a way to show our form types one by one. Now to do so:

```
    {{ form_start(form) }}
        <div>
            Fullname: {{ form_row(contact_us.email) }}
        </div>
        <div>
            {{ form_row(contact_us.email) }}
        </div>
        <div>
            {{ form_row(contact_us.message) }}
        </div>
    {{ form_end(form) }}
```

If you miss to show a field, form_end by default append the unshown fields at the end

To control this by doing the following:

```
    {{ form_end(form, {render_rest: false}) }}
```

You can take a look at the profiler: it gives you informations about the forms used in the page.

To populate the form with data, like in the case of update, we can pass an argument **data** to *createForm* method like so:

```
$form = $this->createForm(type: ContactUsType::class, data: [
    "fullname" => "MNouad Nassri"
]);
```

Now to add bootstrap, go to base.html.twig and add cdn in the stylesheet block

Note: when you have only onew button for submit, it is discouraged top add it in the list of fortypes within your form object; That is because if you add it there, your form will not be re-usable since the button will attach it to a specific action

Note: In case your form has multiple form submit buttons, in this case you can add them to the form type object like this example:

```
$form = $this->createFormBuilder($task)
    ->add('task', TextType::class)
    ->add('dueDate', DateType::class)
    ->add('save', SubmitType::class, ['label' => 'Create Task'])
    ->add('saveAndAdd', SubmitType::class, ['label' => 'Save and Add'])
    ->getForm();
```

In your controller, use the button's isClicked() method for querying if the "Save and add" button was clicked:

```
if ($form->isSubmitted() && $form->isValid()) {
    // ... perform some action, such as saving the task to the database

    $nextAction = $form->get('saveAndAdd')->isClicked()
        ? 'task_new'
        : 'task_success';

    return $this->redirectToRoute($nextAction);
}
```

Validation: Constraints

You can see all validations in symfony by going to docs: symfony validation(supported constraints)

Example: message should not be empty (NotBlank)

Extra notes before begin:

attr that we used to add attributes to inputs, we can do the same to add attributes to the whole form (\ element), and that by adding it to configureOptions:

```
public function configureOptions(OptionsResolver $resolver): void
{
    $resolver->setDefaults([
        'csrf_protection' => true,
        'csrf_field_name' => 'custom_token_name',
        'csrf_token_id'   => 'task_item',
        'attr' => [
            "novalidate" => true,
        ]
    ]);
}
```

With novalidate => true, validation in frontend is disabled so you can easily see how backend validatiuon behaves

Validating a form is done in 2 specific steps:

1. specify constraints of form validation

2. Validate our form